

Corso di Laurea Magistrale in Ingegneria Informatica - A. A. 2023/2024
Ingegneria del Software

Progetto O.C.M.
Open Clinic Manager



Studente:

Salvatore Alonzo (Matricola: 1000052471)

Prefazione

Il seguente documento presenta una descrizione relativa allo sviluppo dell'applicativo Open Clinic Manager implementata durante il corso di Ingegneria del Software. Il software è stato sviluppato in linguaggio Java sfruttando l'ambiente di sviluppo Eclipse, mentre la progettazione in UML è stata realizzata con il software Astah Professional. Verranno qui presentate solamente le versioni finali di ciascun elaborato ottenute al termine di tutte le fasi di progettazione.

Infine vengono descritte le fasi conclusive di testing e di refactoring del programma, con cui sostanzialmente sono state apportate delle migliorie al codice. I dati sono immagazzinati in un database il quale permette la gestione persistente all'operatore.

Indice

1 Ideazione e analisi dei requisiti	pag. 5
1.1 Introduzione	pag. 5
1.2 Requisiti	pag. 5
1.3 Obiettivi e casi d'uso	pag. 5
1.4 Modello dei casi d'uso	pag. 7
1.5 Specifiche Supplementari.	pag. 10
1.6 Glossario	pag. 10
2 Analisi Orientata agli Oggetti	pag. 11
2.1 Introduzione	pag. 11
2.2 Modello di Dominio	pag. 12
2.3 SSD e Contratti	pag. 13
3 Progettazione	pag. 15
3.1 Diagramma delle Classi	pag. 15
3.2 Diagrammi di Sequenza	pag. 17
4 Testing	pag. 18
4.1 Introduzione.	pag. 18
4.2 Individuazione dei casi di test e Testing Unitario	pag. 18

1 Ideazione e analisi dei requisiti

1.1 Introduzione

Lo scopo della fase di ideazione è quello di effettuare un'indagine al fine di ricavare un'idea generale del progetto da realizzare e quindi ottenere delle stime di fattibilità tenendo conto dei tempi e delle risorse necessari.

Per capire al meglio come sviluppare e comprendere i requisiti del progetto, nella fase di ideazione sono stati presi in considerazione i seguenti documenti: Modello dei Casi d'Uso, Documento di Visione, Specifiche Supplementari, Regole di Business e Glossario.

1.2 Requisiti

Il titolare di una clinica richiede la realizzazione di un software che permetta la

Gestione delle prenotazioni della stessa, implicando una gestione implicita di pazienti e medici. Il software deve rappresentare uno strumento integrato che segua il processo di prenotazione in tutte le sue fasi, a partire dalla gestione dei medici e pazienti.

In particolare l'operatore di sala deve poter:

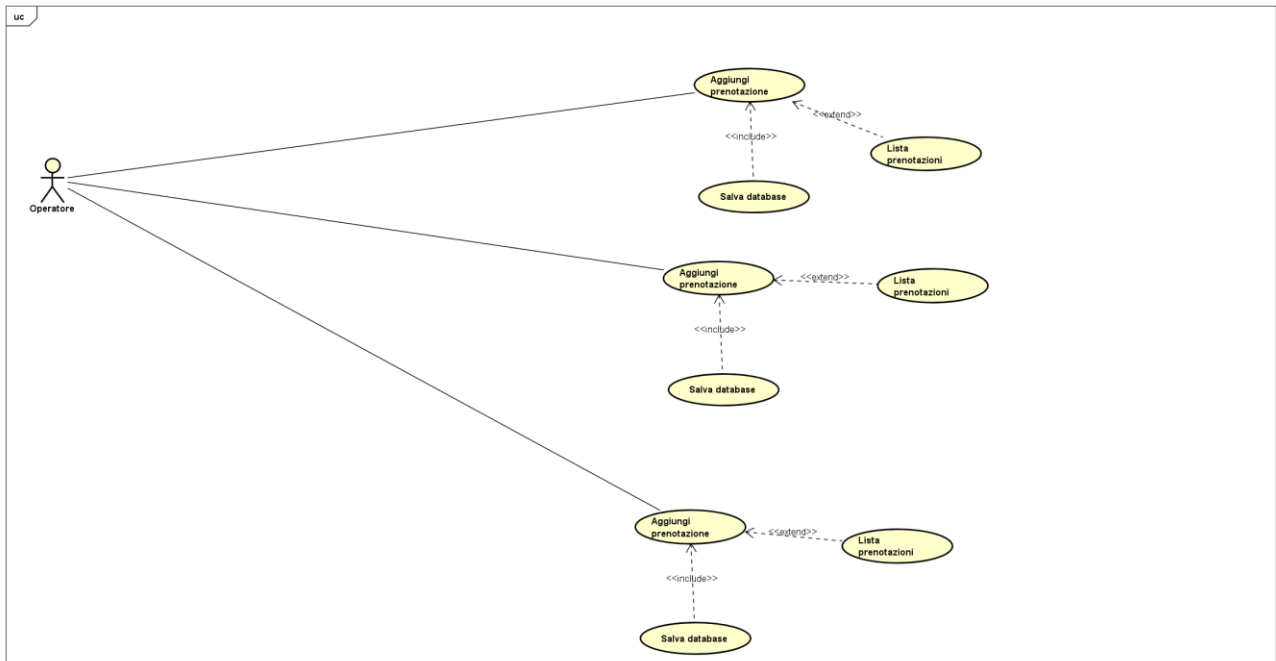
- Anagrafare il paziente
- Anagrafare il medico
- Gestire prenotazioni del paziente
- Consultare una lista di tutti i pazienti della clinica
- Consultare una lista di tutti i medici presenti nella clinica
- Consultare la lista delle prenotazioni di ogni singolo paziente
- Salvare i dati acquisiti dei pazienti in un database (dati persistenti)
- Salvare i dati acquisiti dei medici in un database (dati persistenti)
- Salvare i dati acquisiti nelle varie prenotazioni registrate in un database (dati persistenti)

1.3 Obiettivi e casi d'uso

Analizzando i requisiti riportati nel paragrafo precedente, sono stati individuati l'attore principale a cui è destinato il sistema e gli obiettivi che egli intende portare a termine; da queste informazioni infine sono stati ricavati i casi d'uso principali.

Attore	Obiettivo	Caso d'Uso
Operatore	Anagrafare il paziente	UC1: Inserimento nel database di un nuovo paziente
Operatore	Anagrafare il medico	UC2: Inserimento nel database di un nuovo medico
Operatore	Gestire prenotazioni del paziente	UC3: Inserimento nel database di una nuova prenotazione
Operatore	Consultare una lista di tutti i pazienti della clinica	UC4: Consulto dei pazienti (CRUD, Ricerca Paziente)
Operatore	Consultare una lista di tutti i medici presenti nella clinica	UC5: Consulto dei medici (CRUD, Ricerca Medico)
Operatore	Consultare la lista delle prenotazioni di ogni singolo paziente	UC6: Consulta prenotazioni (CRUD, Ricerca Prenotazioni)
Operatore	Salvare il database dei pazienti	UC7: Salva database pazienti
Operatore	Salvare il database dei medici	UC8: Salva database medici
Operatore	Salvare il database delle prenotazioni	UC9: Salva database prenotazioni

I casi d'uso appena descritti si traducono graficamente nel seguente diagramma UML:



1.4 Modello dei casi d'uso

Viene qui presentata una descrizione dei casi d'uso; in particolare, partendo dalla fase di ideazione in cui la maggior parte dei casi d'uso erano descritti in formato breve, seguendo il processo iterativo si è passati gradualmente ad una descrizione via via più dettagliata per la maggior parte dei casi d'uso, dunque si ha:

UC1: Inserimento nel database di un nuovo paziente

Nome del caso d'uso	Inserimento nel database di un nuovo paziente
Portata	Applicazione
Livello	Obiettivo utente
Attore primario	Operatore
Parti interessate e Interessi	- Operatore: Riporta in un form i dati acquisiti del paziente - Paziente: Fornisce i dati
Precondizioni	Dati del paziente
Garanzia di successo	Il paziente fornisce tutti i dati richiesti dall'operatore

Scenario principale di successo	1. Il paziente ha un problema e richiede una visita con un medico specialistico 2. L'Operatore tramite l'applicativo O.C.M. registra il paziente 3. L'operatore verifica che vi sia disponibilità di un medico per la problematica del paziente 4. L'operatore assegna uno slot temporale del medico specialista al paziente
Estensioni	
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Dipendente dal numero di pazienti
Varie	

UC2: Inserimento nel database di un nuovo medico

Nome del caso d'uso	Inserimento nel database di un nuovo medico
Portata	Applicazione
Livello	Obiettivo utente
Attore primario	Operatore
Parti interessate e Interessi	• Operatore: Riporta in un form i dati acquisiti del medico • Medico: Fornisce i dati
Precondizioni	Dati del medico
Garanzia di successo	Il medico fornisce tutti i dati richiesti dall'operatore
Scenario principale di successo	Nessuno in particolare
Estensioni	
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Dipendente dal numero dei medici
Varie	

UC3: Inserimento nel database di una nuova prenotazione

Nome del caso d'uso	Inserimento nel database di una nuova prenotazione
Portata	Applicazione
Livello	Obiettivo utente
Attore primario	Operatore
Parti interessate e Interessi	• Operatore: Assegna al paziente il medico specialista più indicato alla risoluzione della problematica del paziente • Paziente: Chiede una visita specialistica
Precondizioni	Medico specialista disponibile, Dati del paziente
Garanzia di successo	Presenza di un medico specialista
Scenario principale di successo	1. Il paziente ha fornito i suoi dati e richiesto una visita specialistica 2. L'operatore verifica la presenza dei dati del paziente e dei medici specialisti 3. Una volta effettuate le opportune verifiche, l'operatore assegna un medico specialista opportuno alla richiesta del paziente 4. L'applicativo verifica che il medico sia disponibile per lo slot richiesto dal paziente 5. Una volta effettuata la verifica si assegna lo slot al paziente
Estensioni	
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Dipendente dal numero di pazienti e dalle visite che essi devono affrontare
Varie	

1.5 Specifiche Supplementari

Usabilità

- L'interfaccia grafica deve essere intuitiva e semplice anche per un utente non esperto.
- L'interazione con il sistema non deve presentare un elevato grado di complessità.

Affidabilità

- Il software sviluppato deve essere affidabile e deve poter mantenere i propri dati anche in caso di guasti (guasti elettrici, usura dell'hardware, attacchi informatici).
- Deve essere possibile pianificare dei backup periodici del database la cui cadenza sarà definita più avanti nel progetto.

Vincoli hardware e software

- Per eseguire il software non ci sono particolari requisiti per il sistema operativo (è preferibile utilizzare Windows) purché sia presente la Java Virtual Machine a 64 bit con versione maggiore di 1.8.

Vincoli di sviluppo del software

- Tutto il software è scritto usando Java e utilizza i file in formato json per mantenere i dati persistenti composti da 3 database separati.

Aspetti legali

- Le tecnologie utilizzate per la progettazione e realizzazione del sistema proposto sono di tipo open source o freeware.
- O.C.M. viene rilasciato con licenza open source GPL v3.

1.6 Glossario

Vengono qui riportati i termini più significativi e le relative definizioni.

- **Aggiungi:** Il termine si riferisce al processo di aggiunta da parte dell'operatore di un elemento appartenente ad una delle tre categorie descritte sopra.
- **Lista:** Il termine si riferisce al processo di stampa a video da parte dell'operatore di tutti gli elementi appartenenti ad una delle tre categorie descritte sopra
- **Salva:** Il termine si riferisce al processo di salvataggio in maniera persistente dei dati estrapolati dai vari elementi appartenenti alle tre categorie.

2 Analisi Orientata agli oggetti

2.1 Introduzione

L'approccio seguito per il progetto atto a delimitare le operazioni ripetitive è stato articolato su 3 iterazioni fondamentali, ciò limita la causa di errori durante la progettazione del software.

Le problematiche gestite sono le seguenti:

1. Iterazione 1:

- 1.1 implementazione scenario principale → gestione dell'aggiunta di un nuovo elemento UC1: Inserimento nel database di un nuovo paziente.
- 1.2 implementazione di un caso d'uso → permette di inserire gli elementi necessari alla creazione di un nuovo paziente.

2 Iterazione 2:

- 2.1 implementazione scenario principale → Stampa a video degli elementi appartenenti al database: UC4: Consulto dei pazienti
- 2.2 implementazione di un caso d'uso → permette di stampare a video gli elementi del database

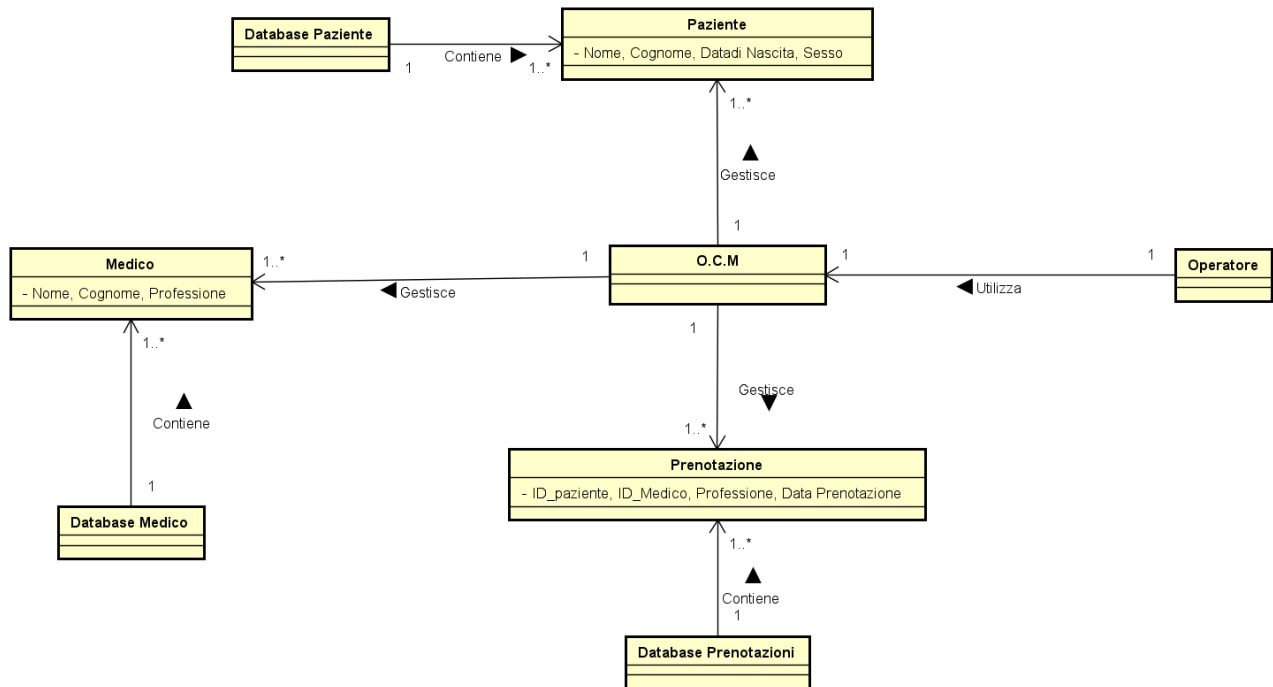
3 Iterazione 3:

- 3.1 implementazione scenario principale → Salvataggio su dispositivo gli elementi appartenenti al database: UC7: Salva database pazienti
- 3.2 implementazione di un caso d'uso → permette di salvare gli elementi in memoria in maniera persistente per i vari database

Tale ricerca delle iterazioni è stata effettuata analizzando i requisiti posti dal cliente, una volta individuati tramite creazione di un modello di dominio, SSD (Sequence System Diagram) e Contratti delle operazioni

2.2 Modello di Dominio

La rappresentazione del modello di Dominio è la seguente:



Le principali classi concettuali sono Le seguenti:

- *Paziente*: Rappresenta il paziente riportato dall'operatore
- *Database Paziente*: Rappresenta il database contenente le informazioni di ogni paziente
- *Medico*: Rappresenta il medico riportato dall'operatore
- *Database Medico*: Rappresenta il database contenente le informazioni di ogni medico
- *Prenotazione*: Rappresenta la prenotazione riportata dall'operatore
- *Database Prenotazione*: Rappresenta il database contenente le informazioni di ogni prenotazione assegnata
- *O.C.M.*: Rappresenta il sistema Opensource Clinic Manager
- *Operatore*: Rappresenta l'attore primario, il quale interagisce con l'applicativo

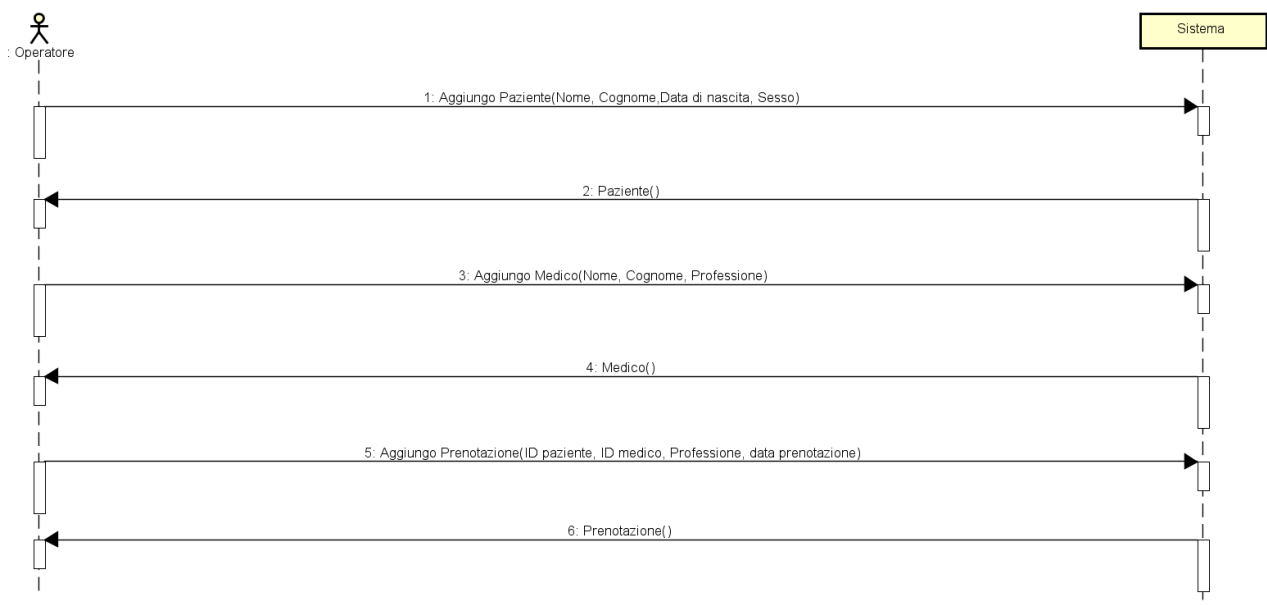
2.3 SSD e Contratti

Procedendo con l'analisi Orienta agli Oggetti, il passo successivo è la creazione del Diagramma di Sequenza di Sistema (SSD) al fine di illustrare il corso degli eventi di input e di output per i vari casi d'uso esaminati in ciascuna iterazione. Inoltre le principali operazioni di sistema individuate negli SSD verranno descritte attraverso i Contratti, quindi avremo.

Le varie iterazioni sono ripetitive per i vari casi, anche ne viene rappresentata una che ne rappresenta anche le successive:

- Iterazione 1

Scenario semplificato del caso d'uso generale riguardo il funzionamento complessivo del software:



Contratto CO1: *Aggiungo Paziente*

Operazione: Aggiungo Paziente (Nome : PazienteBuilder, Cognome : PazienteBuilder ,Data di nascita : PazienteBuilder, Sesso : PazienteBuilder)

Riferimenti: caso d'uso: Gestisco la prenotazione del paziente

Precondizioni: Il paziente necessita di una visita con uno specialista

Post Condizioni: L'operatore inserisce le informazioni necessarie alla registrazione del paziente

Contratto CO2: *Aggiungo Medico*

Operazione: Aggiungo Medico (Nome : MedicoBuilder, Cognome : MedicoBuilder, Professione : MedicoBuilder)

Riferimenti: caso d'uso: Gestisco la prenotazione del paziente

Precondizioni: Il paziente necessita di una visita con uno specialista

Post Condizioni: L'operatore inserisce le informazioni necessarie alla registrazione del medico

Contratto CO3: *Aggiungo Prenotazione*

Operazione: Aggiungo Prenotazione (ID paziente : PrenotazioneBuilder, ID medico :

PrenotazioneBuilder, Professione : PrenotazioneBuilder, Data prenotazione : PrenotazioneBuilder)

Riferimenti: caso d'uso: Gestisco la prenotazione del paziente

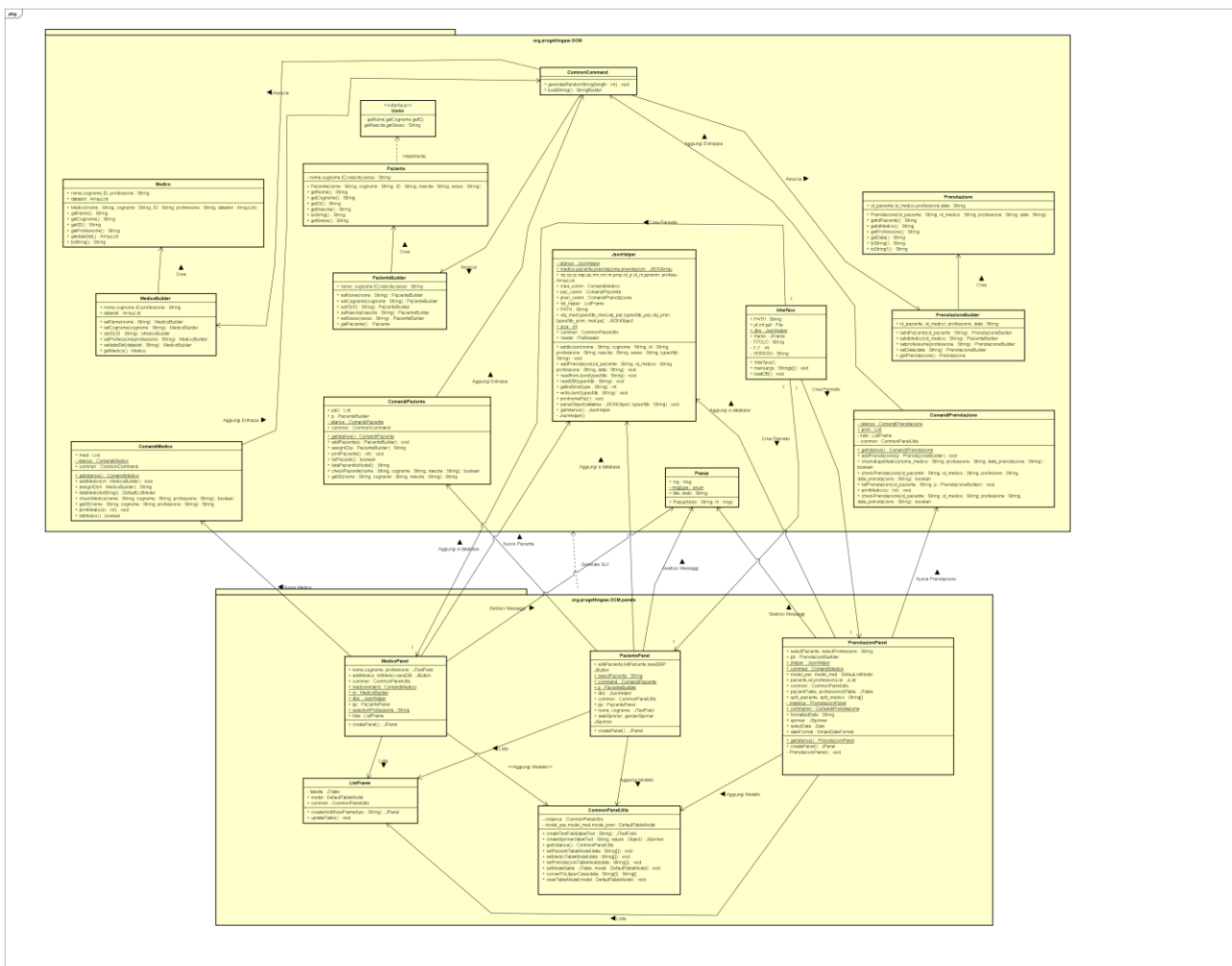
Precondizioni: Il paziente necessita di una visita con uno specialista

Post Condizioni: L'operatore inserisce le informazioni necessarie alla registrazione del medico

3 Progettazione

3.1 Diagramma delle Classi

La progettazione orientata agli oggetti è la disciplina di UP (Unified Process) interessata alla definizione degli oggetti software, delle loro responsabilità e a come questi collaborano per soddisfare i requisiti individuati nei passi precedenti. L'elaborato principale di questa fase è il **Modello di Progetto**, ovvero l'insieme dei diagrammi che descrivono la progettazione logica sia da un punto di vista dinamico (Diagrammi di Interazione) che da un punto di vista statico (Diagramma delle Classi). Il diagramma delle classi viene rappresentato nel seguente modo:



Da questo modello notiamo le classi individuate per il progetto, suddivise in due package per una gestione migliore del codice:

1- Package principale

1. Le classi di comando suddivise in questo modo, rappresentano i comandi che si possono eseguire con gli oggetti creati:
 - 1.1 ComandiMedico
 - 1.2 ComandiPaziente
 - 1.3 ComandiPrenotazione
 - 1.4 CommonCommand
- 2 Interface, rappresenta la parte grafica del software
- 3 JsonHelper, rappresenta la parte di gestione dei database persistenti
- 4 Le classi builders, utilizzate per creare i vari oggetti:
 - 4.1 MedicoBuilder
 - 4.2 PazienteBuilder
 - 4.3 PrenotazioneBuilder
- 5 Medico, descrive l'oggetto Medico
- 6 Paziente, descrive l'oggetto Paziente
- 7 Prenotazione, descrive l'oggetto prenotazione
- 8 Interfaccia Uomo

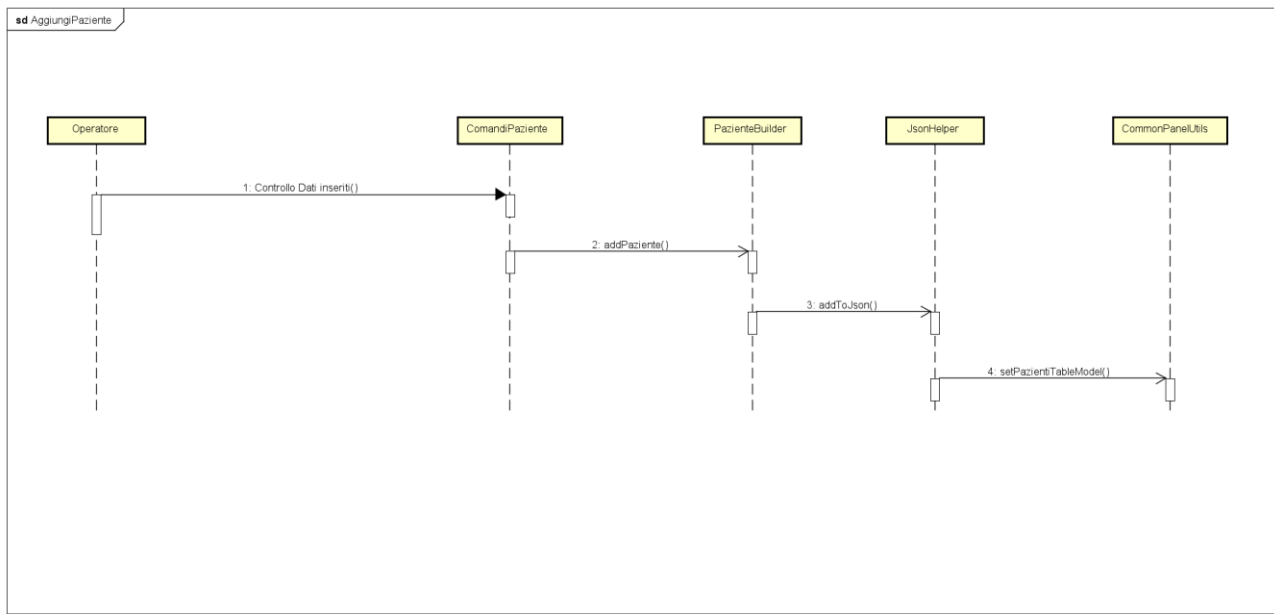
2- Package "Panels"

In questo package sono contenuti i pannelli grafici suddivise per le varie sezioni dedicate ai vari oggetti, suddivisi nel seguente modo:

- 1- CommonPanelUtils, raggruppa le funzioni comuni ai vari panels
- 2- MedicoPanel, qui viene generato il pannello relativo alla gestione dei medici
- 3- PazientePanel, qui viene generato il pannello relativo alla gestione dei pazienti
- 4- PrenotazioniPanel, qui viene generato il pannello relativo alla gestione delle prenotazioni
- 5- ListFrame, qui vengono generate le varie frame relative ai dati inseriti

3.2 Diagrammi di Sequenza

In questo paragrafo viene mostrato il diagramma di sequenza generalizzato per le varie iterazioni, qui possiamo notare la sequenza per l'aggiunta di un Paziente :



4 Testing

4.1 Introduzione

Il testing è una fase essenziale del processo di sviluppo di un programma robusto ed Efficiente, ne permette inoltre la riduzione in maniera sostanziale dei costi di manutenzione. La probabilità di rilevare malfunzionamenti è proporzionale al numero di test eseguiti, tuttavia è difficile testare un programma completamente, infatti le possibili combinazioni di valori di input da prendere in considerazione sono molteplici e non sempre possono essere riprodotti in un tempo ragionevole. Tuttavia un testing progettato accuratamente può rivelare comportamenti anomali ed indesiderati al fine di rendere il software realizzato funzionante e funzionale secondo le specifiche del committente.

I test si dividono principalmente in due famiglie:

- I test unitari (Unit Test) → Sono semplici test/prove che vanno a verificare la correttezza direttamente del codice, in ogni sua piccola parte. L'idea dello Unit Test in Java è quella di valutare ogni singolo metodo in funzione dei valori attesi. Esistono diversi tool per Unit Testing per i diversi linguaggi di programmazione, in particolare JUnit è il framework più diffuso attualmente per l'automazione del testing di unità di programmi Java.
- I test funzionali → Sono dei test che vanno a verificare che il sistema software nella sua completezza funzioni correttamente. Questi test trattano il sistema come se fosse una scatola nera alla quale danno degli input e verificano la correttezza degli output. Per eseguire il testing dell'applicazione realizzata, si è scelto di concentrarsi principalmente su test unitari eseguiti tramite l'ausilio di JUnit.

4.2 Individuazione dei casi di test e Testing Unitario

In una fase preliminare al testing, si è presa coscienza ed ispezione del codice sorgente scritto al fine di operare una scelta sulle classi e sui metodi da testare. Il criterio scelto per individuare tali metodi su cui concentrare i test di unità è stato quello di dare priorità a tutti i metodi che si occupano dell'inserimento di nuove istanze le quali possono facilmente portare ad errori quantità, alla gestione del database e la creazione dei vari oggetti. A seguito dell'esecuzione di questi test unitari con JUnit, si è passati a correggere le relative porzioni di codice che portavano al fallimento di alcuni test.

4.3 Test di Sistema

In previsione di una fase di refactoring in cui verrà integrato un Database con l'applicazione esistente, si è scelto di eseguire dei test di sistema manuali lanciando l'applicazione e provando a portare a termine i principali casi d'uso con i rispettivi scenari alternativi. A seguito di tali test non sono emersi grandi errori nel funzionamento e le prestazioni del sistema si sono mantenute piuttosto elevate, tuttavia sono state rilevate delle imperfezioni e anomalie nell'esecuzione di alcune funzionalità:

Per esempio, sono state individuate alcune implementazioni più ostiche quali la persistenza dei dati per tanto si è provveduto a testarne l'aggiunta dei vari oggetti, la scrittura su file,

la lettura da file e la lettura dei vari oggetti con conseguente pulizia finale per non incombere a malfunzionamenti o oggetti indesiderati durante il funzionamento.